D05 · IA·흐름도

Convoy

명령어 트리 · 핵심 흐름 · 예외 흐름

문서 버전	1.0
작성일	2026-10-20
작성자	김태민
문서 상태	기획 검토본

변경 이력

버전	일자	내용	작성자
1.0	2026-10-20	최초 작성	김태민

목차

1. 명령어 트리
2. 핵심 흐름 3가지 — 실행 · 승인 · 복구
3. 예외 흐름 3가지 — 환경 실패 · 병합 충돌 · 비용 상한

이 문서의 역할 — Convoy 에는 화면이 아니라 **명령**이 있다. 그래서 정보 구조(IA)를 "어떤 명령이 있고 어떤 입구에서 어떻게 상태에 닿는가"로 그리고, 사람이 실제로 거치는 순서를 흐름도로 보인다. 흐름의 상태 이름과 전이는 D04(`states.yaml`), 규칙의 이유는 D03 정책서에서 가져왔다 — 이 문서가 새 규칙을 만들지 않는다. 화면 모양은 D06 장면 목업이 맡는다.

표기 원칙 — `claude -p` 의 동작(결과 JSON · 실패 형태 · 도구 제한 · 고아 작업자 · 플러그인 명령과 배경 실행)은 2026-10-20 구현 1단계에서 실측해 반영했다(D08). 그때 재지 못한 것 — 대표 세션이 설명하는 세부 동작, 사용량 한도 · 디스크 부족의 실제 문구 등 — 에만 "구현 1단계 확인 필요"를 남긴다. 흐름 노드의 `CMD-nn` 은 명령(D07), `SN-nn` 은 장면(D06), `ST-T-nn` · `ST-C-nn` 은 Task · Convoy 상태(D04)다.

1. 명령어 트리

1-1. 두 입구, 하나의 core

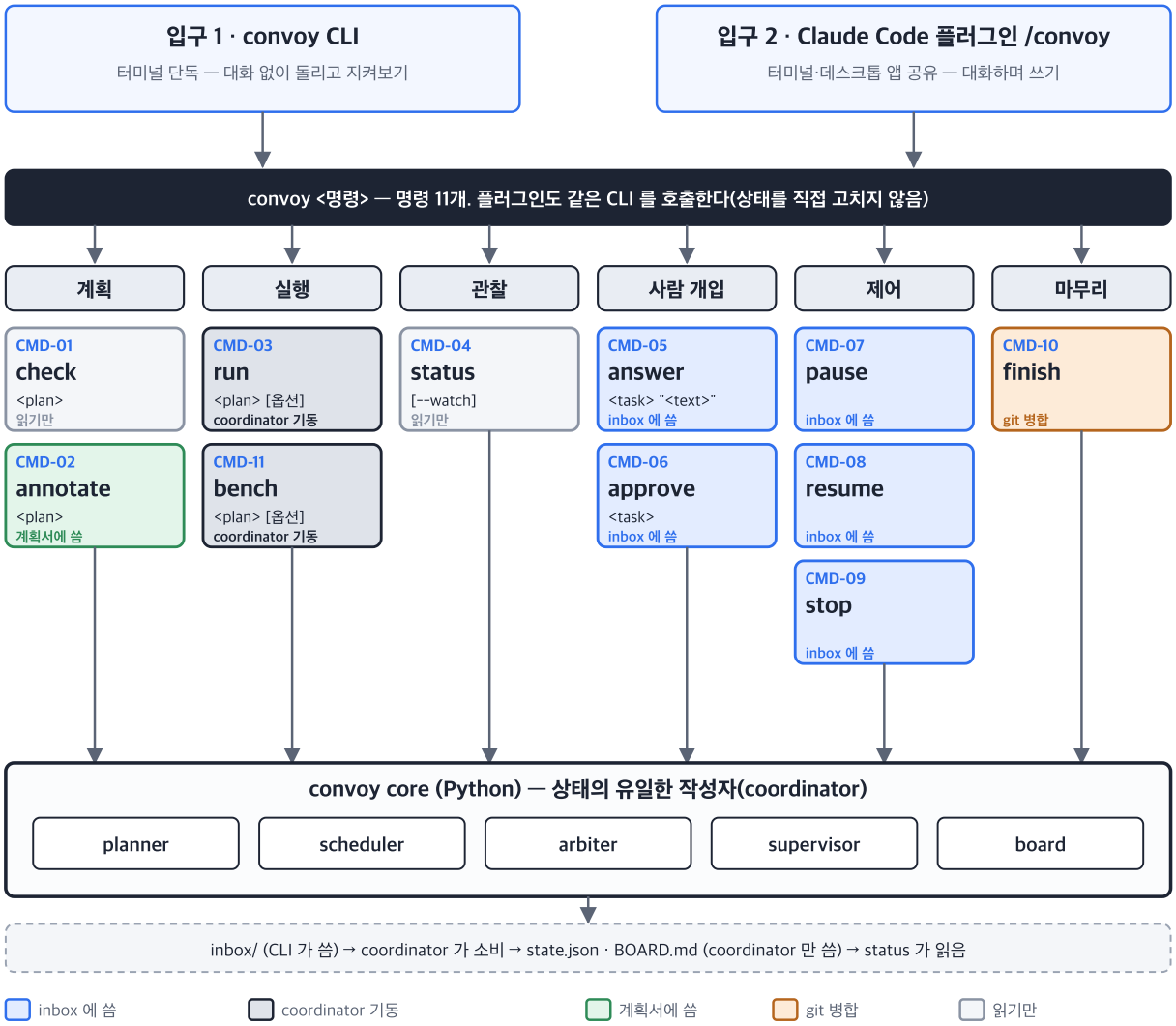


그림 1. 두 입구와 명령어 트리 — 쓰는 곳을 색과 글자로 함께 표시

- **두 입구가 같은 core 로 모인다.** CLI 는 터미널에서 직접 부르고, 플러그인 `/convoy` 는 같은 CLI 를 호출해 결과를 대화로 풀어 준다. 플러그인은 상태를 직접 고치지 않는다 — 단일 작성자 원칙(spec §2-1)을 지키는 장치다.
- **상태를 쓰는 사람은 coordinator 하나다.** 사용자 명령은 inbox 에만 쓰고, coordinator 가 소비한 뒤 `state.json` · `BOARD.md` 를 갱신한다. 그래서 어느 입구에서 시작한 실행이든 다른 입구에서 이어 볼 수 있다.
- 앱에서 대표 세션이 BOARD · outbox · 로그를 읽어 설명하는 부분("3번 왜 막혔어?")의 세부 동작은 구현 1단계 확인 필요.

1-2. 명령 일람

"쓰는 곳"은 명령이 무엇을 바꾸는지다. 상태를 바꾸는 명령 대부분(answer · approve · pause · resume · stop)은 inbox 로만 보내고, 실제 전이는 coordinator 가 한다.

ID	명령	인자 · 옵션	입구	쓰는 곳	연결 장면
CMD-01	`check`	`<plan>`	CLI · 앱 (플러그인)	**읽기만** — 계획서를 읽기만 한다. 이론상 단축률은 의존 그래프에서 계산한 값이며 실측이 아니다	SN-01 SN-11
CMD-02	`annotate`	`<plan>`	앱 (플러그인) · CLI	**계획서에 씀** — 제안(LLM)은 읽기 전용이고, 사용자가 승인한 항목만 계획서의 Depends 줄로 기록한다. CLI 는 터미널에서 한 건씩 y/n 으로 묻고 비-TTY 에서는 제안만 출력한다. Depends 줄이 없던 Task 는 승인분을 쓰면 직전 Task 암묵 의존이 사라진다(미리 한 줄 안내). 플러그인에는 아직 없다	SN-14
CMD-03	`run`	`<plan> [--workers N (기본 3)] [--budget USD (기본 없음, 미지정 시 경고 한 줄 — POL-09)] [--detach (검사 뒤 배경으로 떼어 내고 바로 돌아온다)] [--test "<cmd>" (병합 테스트 명령. 없으면 계획서 **Test:** 줄, 둘 다 없으면 테스트 없이 병합 + 상황판 상시 경고 — POL-05)]`	CLI · 앱 (플러그인)	**coordinator 기동** — coordinator 자신이 state.json 과 BOARD.md 의 유일한 작성자다. 사용자 명령 통로(inbox)를 거치지 않는다. 전경에서 도는 coordinator 이며 Ctrl-C 는 상태를 바꾸지 않고 coordinator 만 끝낸다(작업자는 계속 돈다 — convoy resume 으로 이어 간다). --detach 를 주면 검사 뒤 새 세션의 배경 프로세스로 띄우고 한 줄을 출력한 뒤 바로 끝나며, 그 프로세스의 표준오류는 .convoy/logs/run.err 에 남는다(플러그인이 쓴다)	SN-02 SN-11
CMD-04	`status`	`[--watch]`	CLI · 앱 (플러그인)	**읽기만** — state.json · BOARD.md 를 읽기만 한다. 앱에서는 대표 세션이 BOARD·outbox·로그를 읽어 설명한다(대표 세션의 읽기 동작은 측정하지 않음 — 다음 단계)	SN-03 SN-05 SN-06 SN-07 SN-12
CMD-05	`answer`	`<task> "<text>"`	CLI · 앱 (플러그인)	**inbox 에 씀** — needs_human(wait_kind=blocked) 인 Task 에만 받는다(POL-21). 아니면 CLI 가 inbox 에 쓰지 않고 종료코드 1	SN-04 SN-13
CMD-06	`approve`	`<task>`	CLI · 앱 (플러그인)	**inbox 에 씀** — needs_human 이면서 wait_kind 가 retry_exhausted · merge_conflict · merge_test_failed 인 Task 에만 받는다(POL-21)	SN-06 SN-13
CMD-07	`pause`	—	CLI · 앱 (플러그인)	**inbox 에 씀** — Convoy 가 running 일 때만 받는다. 환경 실패로 인한 paused 는 이 명령이 아니라 coordinator 가 스스로 전이한다	—

ID	명령	인자 · 옵션	입구	쓰는 곳	연결 장면
CMD-08	`resume`	—	CLI · 앱 (플러그인)	**inbox 에 씀** — Convoy 가 paused 일 때 inbox 로 전달한다. coordinator 가 죽어 있으면(.convoy/coordinator.lock 을 쥔 프로세스가 없으면) CLI 가 스스로 coordinator 가 되어 state.json 과 outbox 로 복구하고 루프를 돈다. paused 였다면 resume 을 inbox 에 먼저 쓴다 — 죽어 있을 때 inbox 를 거치는 것은 paused 해제뿐이다. coordinator 가 살아 있으면 draining · completed · aborted 에는 CLI 가 inbox 에 쓰지 않고 바로 거부한다(종료코드 1). 죽어 있을 때는 completed · aborted 만 거부하고, draining 은 복구해 그대로 이어 간다(resume 명령 없이 draining 을 계속한다) (POL-17 · POL-21)	SN-05
CMD-09	`stop`	—	CLI · 앱 (플러그인)	**inbox 에 씀** — running · paused · draining 일 때 받는다. 종료 상태가 아닌 모든 Task 를 cancelled 로 만드는 유일한 명령이다	—
CMD-10	`finish`	—	CLI · 앱 (플러그인)	**git 병합** — Convoy 가 completed · aborted 일 때만 받는다. 병합할 커밋 요약을 보여 주고 사용자 확인(y)을 받은 뒤에만 convoy/integration 을 main 에 merge commit(fast-forward 안 함)으로 병합한다(POL-06). main 이 그사이 움직였어도 평소대로 병합하고, 충돌이면 병합을 abort 해 main 을 그대로 둔 채 오류 3줄과 종료코드 1. 미완료 Task 가 있으면 목록을 함께 보여 주고 다시 확인한다. 상태 파일은 쓰지 않는다	SN-08
CMD-11	`bench`	`<plan> [--runs N (기본 3 — 판정에는 각 3회 이상, spec §7-2)] [--workers N (병렬 작업자 수, 기본 3)] [--template DIR] [--out DIR] [--work-dir DIR] [--budget-run USD] [--budget-total USD] [--run-timeout MIN (기본 20)]`	CLI	**coordinator 기동** — 순차(작업자 1)와 병렬(작업자 --workers, 기본 3)을 회차마다 새 복제본(계획서가 든 저장소의 HEAD 를 clone --no-hardlinks 로 복제, --template DIR 이면 DIR 내용으로 새로 만든 저장소)에서 convoy run 으로 기동한다. 원본 저장소는 건드리지 않는다. 회차 순서는 라운드마다 교대한다(홀수 라운드는 순차→병렬, 짝수는 병렬→순차). 회차 끝에 복제본 integration 에서 계획의 Test 명령을 한 번 더 돌려 최종 테스트 통과를 기록하며 convoy finish 는 하지 않는다. 비용 상한은 --budget-run(회차당)·--budget-total(합계), 회차 시간 상한은 --run-timeout(분, 넘으면 중단하고 timeout 으로 기록한 뒤 이어 간다). 환경 실패로 회차가 일시정지하면 남은 회차를 시작하지 않고 조기 중단으로 기록한다. 종료코드는 모든 회차가 completed 이고 조기 중단이 없을 때만 0, 아니면 1. 결과는 --out(기본 bench/results/)의 results.json 과 회차별 산출물(state.json · BOARD.md · coordinator.log)로 남고, 끝나지 않은 회차도 지우거나 다시 돌리지 않는다. 실제 claude -p 를 여러 번 실행해 비용이 드는 명령이라 첫 회차 직전에 비용 경고와 상한을 보여 준다. 측정값은 results.json 에서만 D09 로 간다	—

1-3. 입구 규칙

규칙	내용
같은 명령	CLI 와 앱이 같은 명령 이름을 쓴다. 앱의 `/convoy answer …` 는 CLI 의 `convoy answer …` 와 같은 inbox 파일을 만든다
annotate	의존 제안은 LLM 이 하므로 대화형 승인에 맞는 앱 쪽을 권장한다(spec §6). CLI 로 구현됨(한 건씩 y/n, 비-TTY 는 제안만). 앱 승인 대화는 구현 전. Depends 없던 Task 는 직전 Task 암묵 의존이 사라진다
승인 대상 확인	answer · approve 는 승인 대기가 아닌 Task 면 CLI 가 먼저 거부한다(POL-21). 최종 판단은 coordinator 가 소비 시점에 다시 한다
main 반영	`finish` 만 main 에 닿는다. 커밋 요약을 보여 주고 사용자 확인을 받는다(POL-06)

2. 핵심 흐름 3가지

사용자·명령 | 시스템·상태 전이 | 판단 | 끝·상태 | → 다음 단계 · 점선 칸막이 = 갈림길

2-1. 실행 — check → run → status → finish

사람이 하는 일은 `check` 로 계획을 확인하고, `run` 으로 띄우고, `status` 로 지켜보고, 끝나면 `finish` 로 main 에 반영하는 네 번이다. 가운데 Task 상태 전이는 coordinator 가 한다.



- `check` 는 읽기만 한다. 이론상 단축률은 의존 그래프에서 계산한 값이지 실측이 아니다.
- `finish` 는 Convoy 가 끝난 뒤(completed · aborted)에만 받고, 미완료 Task 가 남아 있으면 그 목록을 함께 보여 주고 다시 확인한다(POL-06).

2-2. 승인 — blocked → answer

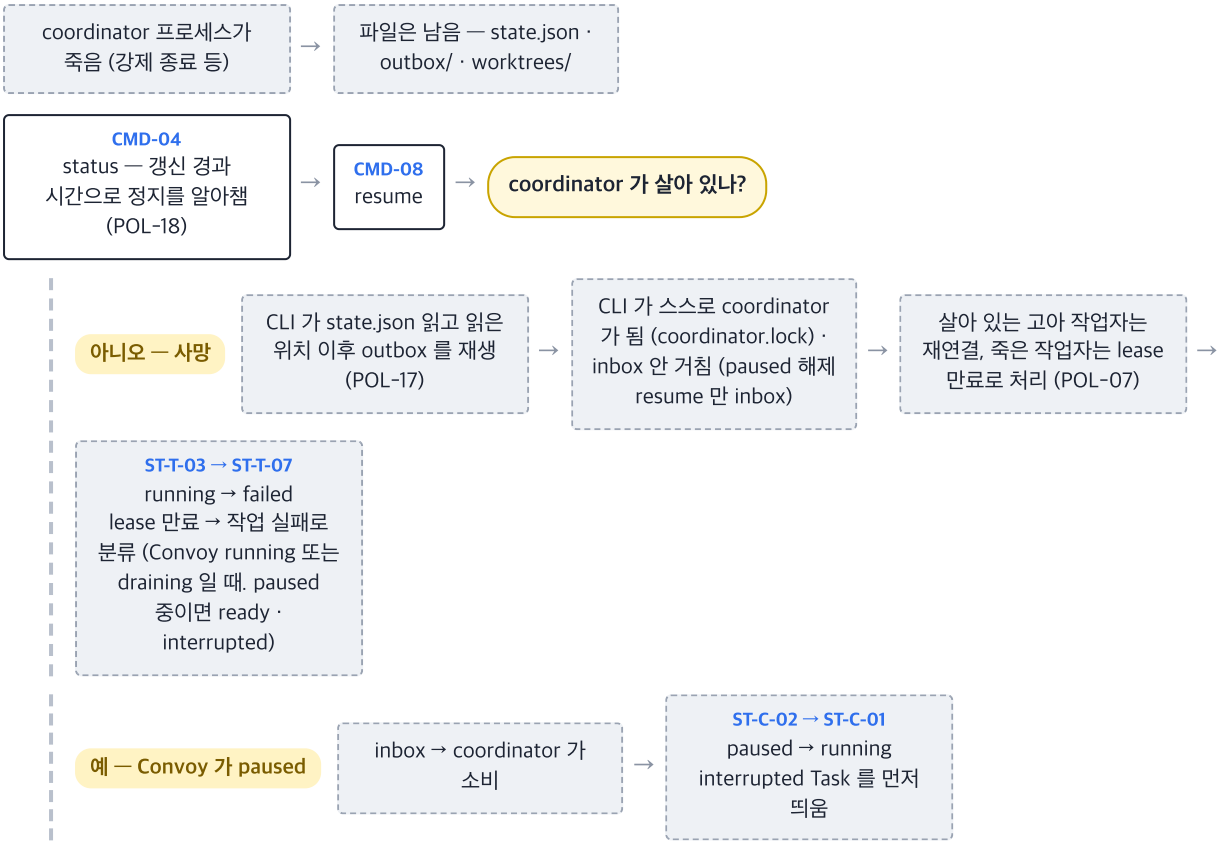
작업자가 사람의 판단이 필요하다고 멈추면 Task 는 needs_human 이 된다. 다른 Task 는 계속 돈다. 사람이 답하거나 승인하면 같은 Task 가 이어진다.



- `answer` 는 작업자가 `blocked` 로 멈춘 Task 에만, `approve` 는 재시도 소진 · 병합 충돌 · 병합 테스트 실패로 멈춘 Task 에만 받는다(POL-21). 앱에서는 대화창에서 같은 명령이 호출된다(SN-13).
- 답은 작업자 프롬프트에 **데이터로** 실린다 — 지시로 해석하지 않는다(POL-15).

2-3. 복구 — coordinator 사망 → resume

상주 서버가 없고 상태가 파일에만 있으므로 coordinator 가 죽어도 파일에서 이어 갈 수 있다(POL-17).



- 같은 `resume` 이 두 일을 한다 — 사용자 일시정지 · 환경 실패로 멈춘 실행 풀기, coordinator 가 죽은 실행 되살리기.
- coordinator 가 죽어 있을 때 `resume` 은 `.convoy/coordinator.lock` 을 쥔 프로세스가 없으면 CLI 가 스스로 coordinator 가 되어 복구하고 루프를 돈다. paused 였다면 resume 을 inbox 에 먼저 쓴다 — 죽어 있을 때 inbox 를 거치는 것은 paused 해제뿐이다.
- 복구 화면(coordinator 사망 → resume)에 대응하는 D06 장면은 없다 — 화면 설계는 D06 범위 밖이며 향후 과제다.

3. 예외 흐름 3가지

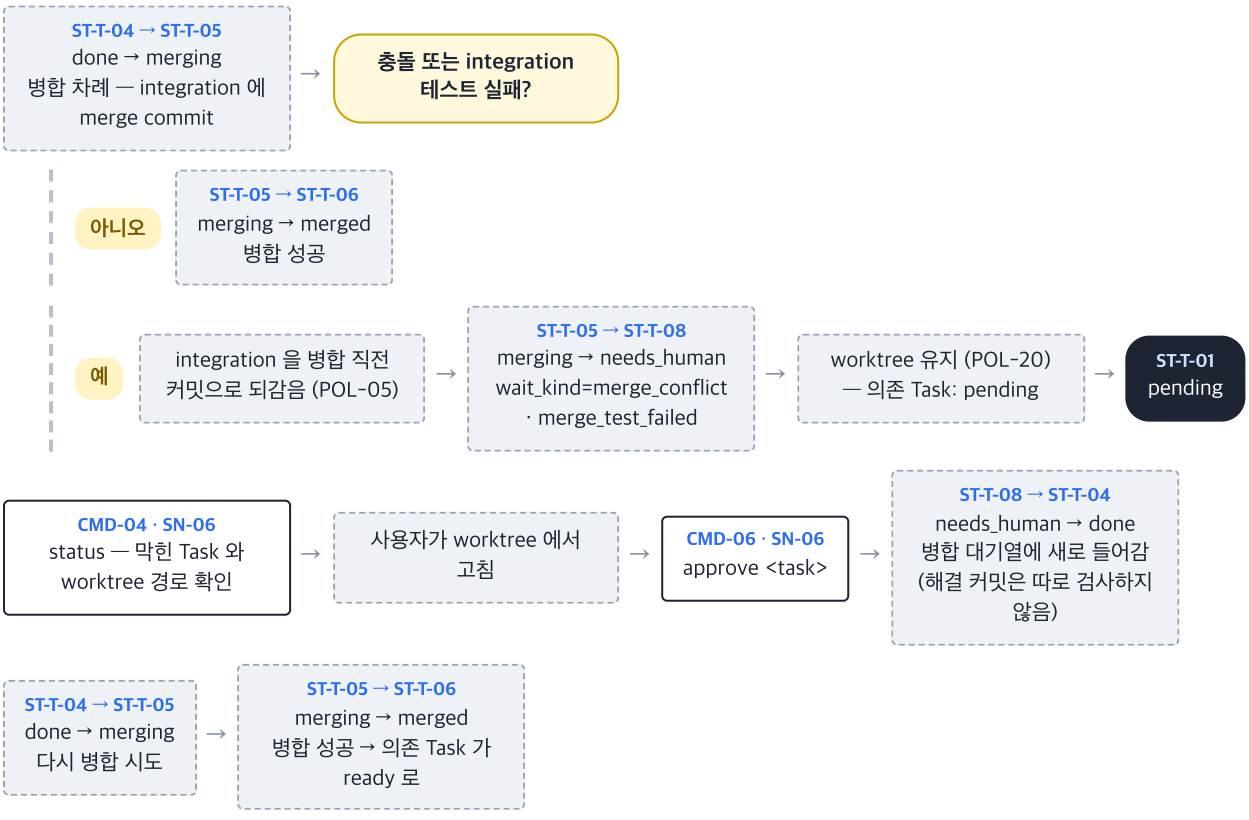
3-1. 환경 실패 — 인증 만료 등

한 Task 의 문제가 아니라 모든 작업자가 같은 벽에 부딪힐 일이므로 Convoy 전체를 멈춘다(POL-11). 환경 실패 신호는 2026-10-20 실측으로 일부 확인했다(인증 없음: 종료코드 1, stderr 빈 값, stdout JSON 의 `is_error` 와 "Not logged in"). 사용량 한도 · 디스크 부족의 실제 문구는 구현 1단계 확인 필요.



3-2. 병합 충돌

병합은 integration 브랜치에 한 번에 하나씩 한다(POL-05). 충돌이나 테스트 실패가 나면 그 병합만 되감고 그 Task 만 needs_human 이 된다 — 다른 Task 는 계속 돈다.



3-3. 비용 상한

곧바로 끊지 않고 draining 으로 마무리한다 — 진행 중 Task 를 끊으면 이미 쓴 비용이 결과 없이 사라진다(POL-09).



- `--budget` 을 지정하지 않으면 상한 없이 실행하되 시작 시 경고 한 줄을 내고 상황판에 누적 비용을 보인다(POL-09).
- paused 중에 진행 중 작업자의 비용 때문에 상한을 넘으면 Convoy 는 paused 그대로이고, `resume` 하면 running 대신 draining 으로 들어간다(D04 전이 규칙).